



Folios — Reliability

FOLIOS · FOLIOSHQ.COM

LAST UPDATED: 2026-06-17

This document describes how Folios stays available, recovers from failure, and protects the user's work. Intended for technical reviewers asking "what happens when something breaks?"

For security controls, see [security.md](#). For architecture, see [architecture.md](#).

Availability posture

Folios runs on managed infrastructure with strong availability SLAs:

Component	SLA	Provider
Static hosting + CDN	99.99%	Vercel
Serverless functions	99.99%	Vercel
Postgres database	99.9%	Supabase Pro tier
Auth	99.9%	Supabase Auth
Realtime	99.9%	Supabase Realtime
AI inference	99.9% (typical)	Anthropic

Folios itself does not publish a formal SLA today — single-user pilot status. The infrastructure SLAs above bound the floor of what Folios can deliver.

Auto-recovery layers

Stale-chunk Lazy import detection

After a deploy, the service worker or HTTP cache can hand the browser an `index.html` whose hashed JS chunks no longer exist on the CDN. Vercel returns the SPA fallback HTML for those 404s, and the browser tries to import HTML as JS. Symptoms:

- `'text/html' is not a valid JavaScript MIME type` (Firefox/Safari)
- `Failed to fetch dynamically imported module` (Chrome)
- `ChunkLoadError / Loading chunk N failed`

Detection happens in three places:

- `window.onerror` listener
- `unhandledrejection` listener
- `ErrorBoundary.componentDidCatch`

`looksLikeChunkReload(message, stack)` matches any of the above patterns. When matched:

1. Log as `chunk_reload` with `resolved: true` (so it doesn't clutter the unresolved Diagnostics list)
2. Dispatch `folio:chunk-reload-detected` event
3. `main.jsx` listens for the event and calls `triggerReload()` immediately

User experience: brief "Updating Folios..." toast, 400ms wait, fresh shell loads. No broken-looking views, no scary errors.

Two redundant SW update paths

Located in `src/main.jsx` :

Path 1: `controllerchange` . Standard service-worker signal that a new SW has taken over. Skips the first event on a fresh visit so new visitors aren't bounced.

Path 2: Version polling. Fetches `/` with `cache: "no-store"` on startup, every 3 min, and on visibility change. Extracts the hashed `index-XXXX.js` filename and compares against the one in the page's loaded `<script src>` . If different, reload. **Completely independent of the service worker** — catches updates even when the SW itself is misbehaving.

Both paths converge on a single `triggerReload()` guarded by a 60-second cooldown so flip-flopping CDN edges during a mid-deploy can't trigger a reload loop.

Realtime reconnection

Supabase Realtime channels drop on network changes. Each domain hook re-subscribes on the next refetch. Connection state is surfaced via the **ConnectionStatus** indicator, which only renders when realtime is dropped (no visual chrome when healthy).

Network resilience

`src/lib/net.js` exposes:

- `pipFetch(url, opts)` — wraps fetch with `AbortController` timeout (30s default for Pip, 10s for general)
- `withRetry(fn, opts)` — exponential backoff retry (used in critical paths)
- `timed(label, fn)` — performance instrumentation
- `pipFetch` handles `429 Too Many Requests` with a polite back-off and surfaces the rate-limit hit to the user

Autosave with localStorage backup

Meeting drafts autosave to Supabase every 1.5 seconds. If the network save fails, the draft is also written to localStorage as a backup. On next page load, the autosave hook checks localStorage and offers to restore. This is a belt-and-suspenders against autosave-network race conditions.

Error observability

Capture surface

Three sources feed `folio_errors` :

1. `window.onerror` + `unhandledrejection` listeners (top-level exceptions)
2. `ErrorBoundary.componentDidCatch` (React render errors)
3. **Explicit** `logError(type, message, opts)` calls from fetch helpers and Pip wrappers

Each captured error includes:

- Error type (`react` | `network` | `pip` | `unhandled` | `rejection` | `chunk_reload`)
- Message
- Stack trace
- Source URL
- User agent
- Optional caller context (action name, account ID, etc.)
- Timestamp

Rate-limited capture

The `logError` function is rate-limited to **20 inserts per minute per user** via in-memory counter, with **60-second dedupe** on identical message hashes. Prevents a render-error loop from flooding the DB.

User-facing surface

Settings → Diagnostics shows the user their own errors with:

- Filters (all / unresolved / this week / this month)
- Per-error expand with full stack + context
- "Copy all" button — formats type, time, URL, message, stack, and context into one clipboard paste for sending to support
- Resolve / Resolve all actions

Auto-recovered errors (`chunk_reload`) are labeled "Auto-recovered" in muted color so they read as known/handled, not alarming.

Render-thrash detection

Not built today (single-user pilot, low volume). Planned for the multi-user phase: render-rate monitor at the App level that flags hooks firing $>N$ times per M seconds.

Data integrity

Row-Level Security at the database

Every user-data table has RLS enforced at the Postgres layer. The database refuses to return rows that don't belong to the requesting user, **regardless of what the application code does**. This is the ultimate backstop.

Cascade deletes

Foreign-key relationships use `on delete cascade` so deleting an account cleanly removes all child rows (meetings, items, contacts, projects, etc.). No orphan rows after deletion.

`gauge_projects.account_id` uses `on delete set null` (per a 2026 fix) so deleting an account doesn't accidentally wipe a tracked project — the project survives, just unlinked.

Schema migrations

Migrations in `supabase/*.sql` are:

- **Idempotent** — `create table if not exists`, `add column if not exists`. Safe to re-run.
- **Single-block** — the entire migration runs as one SQL statement. No partial application.

The canonical schema lives in `supabase/schema.sql` and is kept in sync as the source of truth.

Audit trail

Every write fires `logActivity(action, resource, ...)` into `folio_activity`. Provides a per-user paper trail of every change.

Backup & recovery

Supabase managed backups

- **Daily automated backups** included with the Supabase plan.
- **Point-in-time recovery (PITR)** on Pro tier with RPO < 5 minutes.

- Backups are stored encrypted in Supabase's managed storage.

Recovery procedures

- **Restore to a point in time** — initiated via Supabase Dashboard (Pro tier feature).
- **Single-row restore** — read from a PITR snapshot, manual re-insert.
- **Bulk export/import** — Folios per-account JSON export provides a user-driven backup; not a system-level backup but useful as an escape hatch.

Recovery time / point objectives

Scenario	RPO	RTO
Database corruption	< 5 min (PITR)	Supabase support response time
Accidental delete by user	0 (use restore from PITR)	< 1 hour
Vercel outage	Continuous (CDN cached shell still loads, writes queue)	~minutes (Vercel's typical)
Anthropic outage	N/A (Pip degrades; non-Pip features work)	~minutes

Mid-deploy safety

The "never make Chris clear cache" rule (in CLAUDE.md) drives several deploy-safety controls:

Service worker behavior

- `skipWaiting: true` — new SW activates immediately
- `clientsClaim: true` — takes control of open clients

- `cleanupOutdatedCaches: true` — old caches cleared on activation

Cache headers

`vercel.json` serves these as `Cache-Control: public, max-age=0, must-revalidate :`

- `/`
- `/index.html`
- `/sw.js`
- `/manifest.webmanifest`

Hashed assets stay long-cached (default Vercel behavior).

Reload cooldown

60-second cooldown on `triggerReload()` prevents flip-flopping between two bundle hashes during a mid-deploy CDN-edge propagation window.

Pip-specific reliability

Streaming failures

Streaming responses can stall or fail mid-chunk. The streaming wrapper (`src/lib/pipStream.js`):

- Captures the stream state
- On terminal failure, logs to `folio_errors` with stack
- Surfaces a clear "Pip stream failed" error to the user
- Preserves whatever streamed content was received before the failure

Empty plans

Pip's summarize mode can return an empty plan (nothing to action). This is a valid output, not a failure. The preview modal handles it with a clear "Pip didn't find anything to action here" message and the option to manually add items.

Truncation detection

`looksTruncated(text)` heuristic detects mid-JSON truncation in summarize outputs. When detected, the call is treated as failed rather than producing a broken partial plan. The `max_tokens` for summarize mode was bumped to 3072 specifically to avoid this case on long meeting notes.

Rate-limit handling

Anthropic 429s are caught and surfaced with a "Pip is over capacity" message + retry suggestion.

Mobile & PWA reliability

Offline shell

The service worker caches the app shell. Users can open Folios offline and see the chrome + cached data; writes queue until connectivity returns.

iOS Safari quirks

- All inputs render at ≥ 16 px font (prevents auto-zoom)
- Tap highlight removed
- Overscroll behavior disabled (no rubber-banding)
- Safe-area insets respected on iPhones with notch / island

Multi-device sync

Supabase Realtime subscriptions on every domain hook mean that an edit on one device propagates to other open sessions of the same user within ~500ms (debounce window).

Known reliability gaps

Transparently listed for the multi-user readiness checklist:

- **No formal SLA published** — pilot stage
- **No PagerDuty / on-call alerting** — manual monitoring
- **No multi-region failover** — single Supabase US region
- **No active session management UI** — sign-out is per-browser
- **No render-thrash detector** — planned for multi-user phase
- **No load testing performed** — concurrent-user behavior unproven beyond architectural reasoning
- **No formal incident response playbook** — single-developer judgment today

None of these are blocking single-user pilot. All are on the multi-user readiness list.

Contact

For reliability questions or incident reports: chris.vasconcellos97@gmail.com